Reviewer Pack — Minimal Rank of Noncommutative Crossed Products over Boolean…

Entry dc1ab4fc2155 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 23:22:46 UTC

Minimal Rank of Noncommutative Crossed Products over Boolean Circuit Size

Entry ID: `dc1ab4fc2155` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-25 23:22:46 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Geometry (Crossed Products) **Field B** (complexity object): Complexity Theory: Boolean Circuit Complexity

Statement:

> {'text': 'For any homogeneous polynomial f in noncommutative variables x_1, ..., x_n with coefficients in a field F of characteristic zero, define the minimal rank ρ_f as the minimal number of generators for the ideal generated by all polynomials vanishing on the zeros of f. There exists a constant c > 0 such that for all n ≥ 2, the minimal rank of the polynomial corresponding to a size-n Boolean circuit is upper bounded by ρ_f(c^n) + O(n).', 'quantitative': 'E[ρ_circuit(n)] = Θ(ρ_f(c^n)) + O(n)'}

Rationale (proposer's reasoning):

> {'text': 'The minimal rank of the ideal generated by a polynomial corresponds to a geometric invariant in noncommutative geometry, which has been used to study algebraic structures and their representations. Boolean circuits can be interpreted as noncommutative polynomials in the language of cross products, thus connecting noncommutative geometry with circuit complexity.', 'explanation': 'This bridge may expose new structural insights into both fields by revealing hidden relationships between the algebraic properties of noncommutative spaces and the computational hardness of Boolean functions.'}

Taxonomy category: `GCT_DET_PERM` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `b086338a81f7bca1`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all n ≥ 2 and using 30 random seeds, the correlation coefficient between the size of the Boolean circuits and the computed minimal rank values is greater than or equal to 0.8, with a p-value ≤ 0.05.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 1 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `'noncommutative geometry crossed products' AND 'Boolean circuit complexity'` - `'min rank noncommutative crossed products' AND 'Boolean circuit size'` - `'ideal generators Boolean circuits' AND 'noncommutative variables'`

Top relevant hits considered: - [s2:10.1142/s0129054124500217] Conditionally accepted sampling: AB15 IO scheme with smaller deviation ratio

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.6s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(n):
        if n == 1:
            return [random.choice([0, 1])]
        else:
            left = generate_circuit(n // 2)
            right = generate_circuit(n - n // 2)
            return [left, right]

    def construct_polynomial(circuit):
        if len(circuit) == 1:
            return circuit[0] * (1 << (len(circuit) - 1))
        else:
            left = construct_polynomial(circuit[0])
            right = construct_polynomial(circuit[1])
            return [left, right]

    def min_rank(poly):
        if len(poly) == 1:
            return 1
        else:
```

```
            left_rank = min_rank(poly[0])
            right_rank = min_rank(poly[1])
            return max(left_rank, right_rank)

    n = random.randint(5, 40)
    circuit = generate_circuit(n)
    poly = construct_polynomial(circuit)
    rank = min_rank(poly)

    return {
        "metric_name": "min_rank",
        "metric_value": rank,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [
        int(math.sqrt(i)) * (i % 2 + 1) for i in range(30, 60)
    ]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b4c3a54d.py", line 65, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b4c3a54d.py", line 48, in run_trial
    rank = min_rank(poly)
           ^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b4c3a54d.py", line 41, in min_rank
    left_rank = min_rank(poly[0])
```

```
                ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b4c3a54d.py", line 41, in min_rank
    left_rank = min_rank(poly[0])
                ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b4c3a54d.py", line 41, in min_rank
    left_rank = min_rank(poly[0])
                ^^^^^^^^^^^^^^^^^
   [Previous line repeated 2 more times]
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b4c3a54d.py", line 38, in min_rank
    if len(poly) == 1:
       ^^^^^^^^^
TypeError: object of type 'int' has no len()
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means it did not complete the required computations to verify the conjecture. | next: Investigate and fix the crash in the test code to ensure it can run to completion and provide the necessary correlation coefficient and p-value data for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14377
2	preregistration	ollama_remote	glm4:latest	0	0	5650
3	novelty	ollama_remote	glm4:latest	0	0	4678
4	novelty	ollama_remote	glm4:latest	0	0	5777
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16241
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10546
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8300
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12725
9	judge	ollama_remote	glm4:latest	0	0	14008

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 92303 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/dc1ab4fc2155.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71

2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/dc1ab4fc2155.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/dc1ab4fc2155.tar.gz` (if generated)